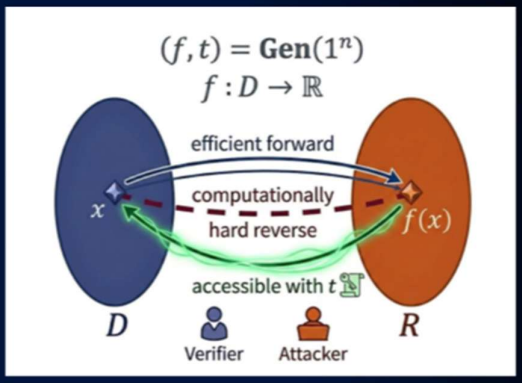
UNIVERSIDAD NACIONAL DE INGENIERÍA

"Ciencia y Tecnología al Servicio del País"

POST QUANTUM CRYPTOGRAPHY

ONE-WAY FUNCTIONS AND TRAPDOORS

- ONE-WAY FUNCTIONS
  - A ONE-WAY FUNCTION IS EASY TO COMPUTE BUT HARD TO INVERT.
  - GIVEN AN INPUT X, COMPUTING F(X) IS EFFICIENT.
  - GIVEN F(X), FINDING X IS COMPUTATIONALLY INFEASIBLE.
- TRAPDOOR FUNCTIONS
  - A TRAPDOOR IS SECRET INFORMATION THAT MAKES INVERSION EASY.
  - WITHOUT THE TRAPDOOR → INVERSION IS HARD.
  - WITH THE TRAPDOOR → INVERSION IS EFFICIENT.



- PUBLIC-KEY CRYPTOGRAPHY RELIES ON TRAPDOOR FUNCTIONS.
  - PUBLIC INFORMATION ALLOWS ENCRYPTION OR VERIFICATION.
  - SECRET INFORMATION ALLOWS DECRYPTION OR SIGNING.

LA FACTORIZACIÓN DE PRIMOS Y EL PROBLEMA RSA

- ACTUALMENTE CASI TODA LA INFRAESTRUCTURA CRIPTOGRÁFICA MUNDIAL SE BASA EN:
  - RSA
  - DIFFIE-HELLMAN
  - ECDH
  - DSA
  - ECDSA
- TODOS ESTOS ALGORITMOS TIENEN ALGO EN COMÚN:
  - SU SEGURIDAD DEPENDE DE PROBLEMAS MATEMÁTICOS CONSIDERADOS DIFÍCILES PARA UNA COMPUTADORA CLÁSICA.

- *LA BASE DEL ALGORITMO RSA :*
  1. *SELECCIONAR DOS NÚMEROS PRIMOS GRANDES p Y q.*
  2. *CALCULAR: $n = p \times q$*
  3. *PUBLICAR n.*
  4. *MANTENER SECRETOS p Y q.*
- *LA SEGURIDAD RADICA EN QUE:*
  - *MULTIPLICAR $p \times q$ ES FÁCIL.*
  - *RECUPERAR p Y q A PARTIR DE n ES EXTREMADAMENTE DIFÍCIL.*
- *ACTUALMENTE NO EXISTE UN ALGORITMO CLÁSICO EFICIENTE PARA FACTORIZAR ENTEROS GRANDES.*
- *POR TANTO, SIGUE SIENDO SEGURO FRENTE A ATACANTES CLÁSICOS.*

ALGORITMO DE SHOR

- EN 1994, PETER SHOR PROPUSO UN ALGORITMO CUÁNTICO CAPAZ DE RESOLVER 2 PROBLEMAS MATEMÁTICOS FUNDAMENTALES EN TIEMPO POLINOMIAL:
  - FACTORIZACIÓN DE NÚMEROS ENTEROS GRANDES.
  - LOGARITMO DISCRETO.
- ESTOS PROBLEMAS CONSTITUYEN LA BASE DE LA SEGURIDAD DE:
  - RSA
  - DIFFIE-HELLMAN (DH)
  - ELLIPTIC CURVE DIFFIE-HELLMAN (ECDH)
  - DSA
  - ECDSA

- COMPUTACIÓN CLÁSICA
  - PARA FACTORIZAR UN NÚMERO GRANDE: 2048 BITS → TIEMPO EXTREMADAMENTE ELEVADO.
- COMPUTACIÓN CUÁNTICA (SHOR)
  - REDUCE EL PROBLEMA A UNA COMPLEJIDAD POLINOMIAL, HACIENDO VIABLE RESOLVERLO CON UNA COMPUTADORA CUÁNTICA SUFICIENTEMENTE GRANDE.
- EL ALGORITMO DE SHOR REDUCE DRÁSTICAMENTE LA DIFICULTAD MATEMÁTICA SOBRE LA CUAL RESIDE LA SEGURIDAD DE LOS ALGORITMOS RSA Y ECC.

NIST PQC STANDARDS
WHY A STANDARDIZATION PROCESS?

- SHOR'S ALGORITHM THREATENS RSA, DSA, ECDSA, DH, ECDH - NEARLY ALL DEPLOYED PUBLIC-KEY CRYPTO.
- THE COMMUNITY ALREADY HAD MANY "QUANTUM-SAFE" CANDIDATES
  - LATTICES.
  - CODES.
  - HASHES.
  - ISOGENIES.
  - MULTIVARIATE.
- BUT:
  - NO AGREED PARAMETERS
  - NO AGREED SECURITY LEVELS.
  - NO COMMON API, NO INTEROPERABILITY.
  - "HARVEST-NOW, DECRYPT-LATER" MAKES THE TIMELINE URGENT.
- A TRUSTED BODY NEEDED TO EVALUATE CANDIDATES, PICK WINNERS, AND PUBLISH REFERENCE PARAMETERS THAT VENDORS CAN IMPLEMENT AGAINST.

REAL-WORLD EXAMPLE: TLS "HARVEST NOW, DECRYPT LATER"

- *TYPICAL TLS 1.3 SESSION TODAY (2026):*
  - *CLIENT AND SERVER PERFORM ECDHE KEY EXCHANGE.*
  - *CLIENT SENDS EPHEMERAL PUBLIC KEY: g^a (WHERE a IS SECRET).*
  - *SERVER SENDS EPHEMERAL PUBLIC KEY: g^b (WHERE b IS SECRET).*
  - *BOTH COMPUTE SHARED SECRET: g^{ab} → DERIVE AES SESSION KEY.*
  - *ALL TRAFFIC ENCRYPTED WITH AES-256-GCM.*
  - *ATTACKER SEES g^a AND g^b ON THE WIRE.*
  - *CANNOT COMPUTE a FROM g^a (DISCRETE LOG IS HARD).*
  - *CANNOT COMPUTE g^{ab} WITHOUT a OR b.*
  - *FORWARD SECRECY PROTECTS PAST SESSIONS.*
- *QUANTUM THREAT (FUTURE ~ 2030s):*
  - *HARVEST NOW (2026): ADVERSARY RECORDS AND STORES:*
    - *CAPTURED g^a, g^b.*
    - *ALL ENCRYPTED TRAFFIC (CIPHERTEXT).*
  - *DECRYPT LATER (~2030s): WITH QUANTUM COMPUTER:*
    - *USE SHOR'S ALGORITHM TO SOLVE DISCRETE LOG.*
    - *COMPUTE a FROM g^a AND p.*
    - *COMPUTE g^{ab} → RECOVER SESSION KEY.*
    - *DECRYPT ALL STORED TRAFFIC.*

NIST STANDARDS AND NAMING CONVENTIONS

FIPS	NIST NAME	ORIGINALLY CALLED	PURPOSE	FAMILY
FIPS 203	ML-KEM	CRYSTALS-KYBER	KEY ENCAPSULATION	LATTICE (MODULE-LWE)
FIPS 204	ML-DSA	CRYSTALS-DILITHIUM	DIGITAL SIGNATURE	LATTICE (MODULE-LWE/SIS)
FIPS 205	SLH-DSA	SPHINCS+	DIGITAL SIGNATURE	HASH-BASED
FIPS 206 (DRAFT)	FN-DSA	FALCON	DIGITAL SIGNATURE	LATTICE (NTRU)
(SEPARATE)	HQC	HQC	BACKUP KEM (MAR 2025)	CODE-BASED

- *ML- = MODULE-LATTICE (MODULE-LWE BASED)*
- *SLH- = STATELESS HASH-BASED*
- *FN- = FOURIER-NTRU (NTRU + FFT SAMPLING)*
- *-KEM = KEY ENCAPSULATION MECHANISM*
- *-DSA = DIGITAL SIGNATURE ALGORITHM*
- *ML-KEM = MODULE-LATTICE BASED KEY ENCAPSULATION MECHANISM*
- *ML-DSA = MODULE-LATTICE BASED DIGITAL SIGNATURE ALGORITHM*
- *SLH-DSA = STATELESS HASH-BASED DIGITAL SIGNATURE ALGORITHM*
- *FN-DSA = FFT-OVER-NTRU DIGITAL SIGNATURE ALGORITHM*

TERMINOLOGY SHIFT: "KEY EXCHANGE" → "KEY ENCAPSULATION"

- *CLASSICAL DIFFIE–HELLMAN (KEY EXCHANGE):*
  - *ALICE AND BOB EACH PUBLISH g^a AND g^b.*
  - *BOTH INDEPENDENTLY COMPUTE THE SAME SHARED SECRET g^{ab}.*
  - *GENUINELY SYMMETRIC - AN EXCHANGE.*
- *POST-QUANTUM KEM:*
  - *ALICE PUBLISHES A PUBLIC KEY. BOB HAS NO PUBLISHED KEY.*
  - *BOB PICKS A RANDOM SESSION KEY K, ENCAPSULATES IT UNDER ALICE'S K_{pub} TO PRODUCE A CIPHERTEXT c.*
  - *ALICE DECAPSULATES c WITH HER SECRET KEY TO RECOVER K.*
  - *CLOSER TO PUBLIC-KEY ENCRYPTION OF A RANDOM SESSION KEY THAN A SYMMETRIC EXCHANGE.*

LATTICE SCHEMES DO NOT GIVE A CLEAN COMMUTATIVE GROUP OPERATION LIKE $g^{ab} = g^{ba}$. THE NATURAL PRIMITIVE IS "ENCRYPT A RANDOM SESSION KEY" - HENCE KEM.

LIBRERÍAS PQC DISPONIBLES ACTUALMENTE

- OPEN QUANTUM SAFE (OQS)
  - FABRICANTE:
    - OPEN QUANTUM SAFE PROJECT https://openquantumsafe.org/
  - CARACTERÍSTICAS:
    - IMPLEMENTA ESTÁNDARES NIST.
    - INTEGRACIÓN CON OPENSSL.
    - SOPORTE PARA ML-KEM Y ML-DSA.
    - PROYECTO ACADÉMICO-INDUSTRIAL LÍDER.
- liboqs
  - FABRICANTE:
    - OPEN QUANTUM SAFE PROJECT
  - LENGUAJES:
    - C
    - C++
    - PYTHON
    - GO
    - RUST
    - JAVA (MEDIANTE BINDINGS)
  - USO RECOMENDADO EN INVESTIGACIÓN.
- OPENSSL 3 + OQS PROVIDER
  - FABRICANTES:
    - OPEN QUANTUM SAFE
    - OPENSSL FOUNDATION
    - https://github.com/open-quantum-safe/oqs-provider
  - PERMITE:
    - TLS HÍBRIDO.
    - CERTIFICADOS PQC.
    - LABORATORIOS REALES DE MIGRACIÓN.
- BOUNCY CASTLE PQC (JAVA)
  - FABRICANTE:
    - BOUNCY CASTLE PROJECT https://www.bouncycastle.org/
  - SOPORTA:
    - ML-KEM (KYBER)
    - ML-DSA (DILITHIUM)
    - FALCON/FN-DSA
    - SPHINCS+
  - VENTAJAS:
    - INTEGRACIÓN DIRECTA CON JCA/JCE.
    - NO REQUIERE COMPONENTES NATIVOS.

UNIVERSIDAD NACIONAL DE INGENIERÍA

¡MUCHAS GRACIAS!